

Project 4 - PowerShell Investigation

Purpose

Learn basic PowerShell navigation and file management.

Commands to Learn

Get-Process	Lists running processes		Get-Help/Get-Command	Displays command help
Get-EventLog	Reads event logs		Get-content	Reads file contents
Get-ChildItem	Lists files/folders		Set-content	Overwrites file contents
New-Item	Creates files/folders		Get-location	Shows current directory
Add-content	Appends to a file		Get-NetTCPConnection	Lists TCP connections

Questions To Answer

- How can PowerShell gather information?
 - How can I view running processes?
 - How can I view Windows event logs?
 - How to navigate PowerShell?
-

My Findings:

Alright, so from what I know about PowerShell (PS), it is very similar to both Linux and Windows Command Prompt (CMD). However, syntax is different and PS tends to be used to automate larger tasks.

The very first thing I did in PS was to try “man” this is how I was able to find information about commands in linux. Here it gave me a weblink I could use to find help if I got stuck

(<http://go.microsoft.com/fwlink/?LinkID=108518>).

This had a very helpful intro list of commands, and I took a few minutes to read through some of these and see what options I had available to me. While reading through these I noticed that the windows version of “man” is “get-help” or “get-command” and provides files, downloads, and websites for additional information.

```
PS C:\Users\allyr> get-command
```

CommandType	Name	Version	Source
Alias	Add-AppPackage	2.0.1.0	Appx
Alias	Add-AppPackageVolume	2.0.1.0	Appx
Alias	Add-AppProvisionedPackage	3.0	Dism
Alias	Add-MsixPackage	2.0.1.0	Appx
Alias	Add-MsixPackageVolume	2.0.1.0	Appx
Alias	Add-MsixVolume	2.0.1.0	Appx
Alias	Add-ProvisionedAppPackage	3.0	Dism
Alias	Add-ProvisionedAppSharedPackageContainer	3.0	Dism
Alias	Add-ProvisionedAppxPackage	3.0	Dism
Alias	Add-ProvisioningPackage	3.0	Provisioning
Alias	Add-TrustedProvisioningCertificate	3.0	Provisioning
Alias	Apply-WindowsUnattend	3.0	Dism
Alias	Disable-PhysicalDiskIndication	2.0.0.0	Storage

The first thing I did was see what information PS could find for me. I started with something easy to verify “Get-NetTCPConnection”. The information this provided was similar to that of my last project in Wireshark. I was able to see open connections, what ports they were using, and the IPV6 associated with them.

Select Administrator: Windows PowerShell

```
PS C:\WINDOWS\system32> Get-NetTCPConnection
```

LocalAddress	LocalPort	RemoteAddress	RemotePort	State	AppliedSetting
2600:1700:3490:7550:bc41:ab4c:90...	64657	2606:4700:4408::ac40:9bd1	443	Established	Internet
2600:1700:3490:7550:bc41:ab4c:90...	64128	2607:f8b0:4002:c08::bc	5228	Established	Internet
2600:1700:3490:7550:bc41:ab4c:90...	63152	2603:1030:f00:f::389	443	Established	Internet
2600:1700:3490:7550:bc41:ab4c:90...	62710	2600:1700:3490:7550::1	53	TimeWait	
2600:1700:3490:7550:bc41:ab4c:90...	62205	2607:f8b0:4002:c2c::6c	993	Established	Internet
2600:1700:3490:7550:bc41:ab4c:90...	61766	2600:1700:3490:7550::1	53	TimeWait	
2600:1700:3490:7550:bc41:ab4c:90...	61352	2603:1063:2001:102::365:ff1	443	Established	Internet
2600:1700:3490:7550:bc41:ab4c:90...	61350	2603:1063:2001:13b::365:ff1	443	Established	Internet
2600:1700:3490:7550:bc41:ab4c:90...	59971	2600:1700:3490:7550::1	53	TimeWait	
2600:1700:3490:7550:bc41:ab4c:90...	59441	2600:1700:3490:7550::1	53	TimeWait	

I saw there was also a command that would connect to my Event Viewer project “Get-EventLog”. Let’s see what that turns up.

```
PS C:\WINDOWS\system32> Get-EventLog
```

```
cmdlet Get-EventLog at command pipeline position 1
Supply values for the following parameters:
LogName: Event Viewer
Get-EventLog : The event log 'Event Viewer' on computer '.' does not exist.
At line:1 char:1
+ Get-EventLog
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [Get-EventLog], InvalidOperationException
+ FullyQualifiedErrorId : System.InvalidOperationException,Microsoft.PowerShell.Commands.GetEventLogCommand
```

Ok, so heads up, it isn’t asking for the application name here, Whoops! Instead, If I wanted to read the logs I had to ask it to first list the logs, then select which log I’d like to investigate.

```
PS C:\WINDOWS\system32> Get-EventLog -list
```

Max(K)	Retain	OverflowAction	Entries	Log
20,480	0	OverwriteAsNeeded	14,153	Application
20,480	0	OverwriteAsNeeded	0	HardwareEvents
512	7	OverwriteOlder	0	Internet Explorer
20,480	0	OverwriteAsNeeded	0	Key Management Service
128	0	OverwriteAsNeeded	1,344	OAuthAlerts
512	7	OverwriteOlder	287	Realtek
20,480	0	OverwriteAsNeeded	31,848	Security
20,480	0	OverwriteAsNeeded	41,184	System
15,360	0	OverwriteAsNeeded	11,051	Windows PowerShell

I decided to take a look at the Security log. WOAHH, ok.... Lesson learned here... not the best way to view this log file. It went through and listed several days' worth of log events and was definitely not as easy to see or maneuver as it was in Event Viewer. I was able to see that it did present the same event IDs but can absolutely see why there is a separate application for that. Viewing them here also didn't provide as much information.

2957860	Jul 03 10:19	SuccessA...	Microsoft-Windows...	4624	An account was successfully logged on....
2957859	Jul 03 10:18	SuccessA...	Microsoft-Windows...	4672	Special privileges assigned to new logon....
2957858	Jul 03 10:18	SuccessA...	Microsoft-Windows...	4624	An account was successfully logged on....
2957857	Jul 03 10:13	SuccessA...	Microsoft-Windows...	4672	Special privileges assigned to new logon....
2957856	Jul 03 10:13	SuccessA...	Microsoft-Windows...	4624	An account was successfully logged on....
2957855	Jul 03 10:08	SuccessA...	Microsoft-Windows...	4672	Special privileges assigned to new logon....
2957854	Jul 03 10:08	SuccessA...	Microsoft-Windows...	4624	An account was successfully logged on....
2957853	Jul 03 10:07	SuccessA...	Microsoft-Windows...	5379	Credential Manager credentials were read....
2957852	Jul 03 10:07	SuccessA...	Microsoft-Windows...	5379	Credential Manager credentials were read....

Ok, I played around a little, now let's see if I can do some of the same things I was able to do in my Linux Basics project. I used the PS version of 'pwd' which is "get-location" and then took a look at what was in this location using "get-childitem". Ok, I'll be honest, I was not expecting there to be so many items here. Upon doing a quick google search I found out that because I am using PS as the administrator, it's showing the contents of the System32 directory, which contains a large portion of Windows' system files.

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-location

Path
----
C:\WINDOWS\system32

PS C:\WINDOWS\system32> get-childitem

Directory: C:\WINDOWS\system32

Mode                LastWriteTime         Length Name
----                -
d-----         11/3/2024   6:22 PM             %userprofile%
d-----         4/1/2024    4:06 AM             0409
d-----         3/28/2025    1:29 AM      AccountHealthAssets
d-----         5/1/2026   12:29 AM      AdvancedInstallers
d-----         6/24/2026   12:29 AM             af-ZA
```

Ok, I want to narrow the scope here, so I am going to attempt to move to my Documents folder and work from there. Using the list that I found earlier using "get-help" I found, "set-location" and tried navigating to my Documents folder. "set-location C:\users\lallyr\documents".

```
PS C:\WINDOWS\system32> set-location C:\users\allyr\documents
set-location : Cannot find path 'C:\WINDOWS\system32\users\allyr\documents' because it does not exist.
At line:1 char:1
+ set-location C:\users\allyr\documents
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\WINDOWS\syst...allyr\documents:String) [Set-Location], ItemNotFou
ndException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand
```

Weird, I got an error saying the path doesn't exist...

Welp, google tells me that's because of syntax. Apparently "c:\users\allyr" and "c:\users\allyr" are different commands. The first one is a path relative to the drive I used, and the second one is an absolute path. Ok, lesson learned. It also provided a faster method to get to my user folder...

```
PS C:\WINDOWS\system32> Set-Location $HOME\Documents
PS C:\Users\allyr\Documents> _
```

Ta-Da! "Set-Location \$HOME\Documents" takes me right there. Alright, finally able to create a new directory. I used "New-item -itemtype Directory PowershellLab", breaking it down, this command is creating a new item, I am specifying what type of item and then naming it.

```
PS C:\WINDOWS\system32> Set-Location $HOME\Documents
PS C:\Users\allyr\Documents> New-item -itemtype Directory PowershellLab

Directory: C:\Users\allyr\Documents
```

Mode	LastWriteTime	Length	Name
d----	7/9/2026 1:11 PM		PowershellLab

I will now go into that directory and create a file. Working inside my own folder made it much easier to understand what each command was doing compared to working inside the Windows System32 directory.

```
PS C:\Users\allyr\Documents> set-location PowershellLab
PS C:\Users\allyr\Documents\PowershellLab> new-item -itemtype file notes.txt

Directory: C:\Users\allyr\Documents\PowershellLab
```

Mode	LastWriteTime	Length	Name
-a----	7/9/2026 1:15 PM	0	notes.txt

```
PS C:\Users\allyr\Documents\PowershellLab> _
```

Now, just like in Linux, there are several ways I can add information to this file. "add-content" will add to the end of a file, which is fine if I want to set a new date or with a cumulative report. "set-content" can

overwrite it entirely. Since I don't currently have anything on the txt file. I will just add text for now and then check that it was updated properly using "get-content".

```
PS C:\Users\allyr\Documents\PowerShellLab> add-content notes.txt "learning to add text"
PS C:\Users\allyr\Documents\PowerShellLab> get-content notes.txt
learning to add text
PS C:\Users\allyr\Documents\PowerShellLab>
```

Day 2-

Copy-Item	Copy
Move-Item	Move
Remove-Item	Deletes files and directories

Lessons Learned

- PowerShell supports Linux-style aliases (such as ls, cd, and pwd).
- PowerShell can read Windows Event Logs, but Event Viewer is often easier for detailed investigations.
- Small syntax differences (such as C:folder vs C:\folder) can completely change how PowerShell interprets a command.
- Creating a personal workspace made learning PowerShell much easier than working directly in System32.